SuperCon 2026 本選レポート

チーム15

1. プログラミングの工夫点

1.1 最大の工夫：「数撃ちゃ当たる」確率的多スタート探索

この問題では、円盤をただ圧縮していくだけでは、円盤間に隙間が残っているにもかかわらず、それ以上動けない状態に陥ることがある。私たちはこの状態を「悪い局所解」と捉え、最終的な成績は圧縮計算の精度だけでなく、どの局所解へ到達するかに大きく左右されると考えた。

そこで、一つの初期配置を極限まで長く圧縮するのではなく、初期配置に小さな摂動を与えた多数の試行を並列に実行し、その中から有望な解を選ぶ方針を採用した。私たちはこれを「数撃ちゃ当たる戦法」と呼んだ。より正式には、**初期摂動を用いた確率的多スタート探索**である。

検証では、同じ入力で摂動の大きさだけを変えた場合と、摂動方向を変えた場合を調べた。その結果、円盤直径の1%程度という小さな初期摂動でも到達する局所解と最終的な L が変わり、摂動を大きくすれば改善量も大きくなるという単純な相関は見られなかった。このため、大きく動かすよりも、小さな摂動を異なる方向へ多数試す方が有効だと判断した。

この方針は、局所解を避ける巧妙な規則を一つ作るのではなく、富岳の余った計算資源を「異なる局所解を探す確率」へ振り向ける逆転的な発想である。私たちの実装で最もアピールしたい点である。

1.2 多数試行のための計算資源配分

富岳の4ノードを、16 MPIプロセス、各12 OpenMPスレッドとして使用した。8問それぞれに2 MPIプロセスを割り当て、異なる乱数系列の初期摂動から独立に探索した。1ノード当たりでは4プロセス×12スレッドとなり、48コアを使用する構成である。

一つの問題を24スレッドで1回だけ計算する方法や、1 MPIプロセスの中で複数試行をネストしたOpenMPで動かす方法も検証した。しかし、ネスト並列にはオーバーヘッドがあり、独立試行はMPIプロセスとして分離し、各試行内部の力計算をOpenMPで並列化する構成の方が効率的だった。これは単なる高速化ではなく、多数の局所解を探索するという方針に合わせた並列化である。

1.3 粗探索と有望解への計算集中

限られた600秒を、約550秒の粗探索と、残り時間の精密化に分けた。各プロセスは異なる初期摂動から試行を繰り返すが、全結果を保存するのではなく上位2候補だけを保持する。粗探索終了後、同じ問題を担当する2プロセスがMPI通信で候補をメモリ上交換し、全候補の中で有望な2解だけを精密に圧縮した。

これにより、悪い局所解を引いた試行を最後まで精密化する時間を減らし、「広く探す段階」と「良い候補へ計算を集中する段階」を両立させた。当初は候補交換をファイルで行っていたが、最終版ではMPI通信へ置き換え、待ち時間と不要なファイル入出力を削減した。

1.4 局所解からのescape

上位候補の精密化中に圧縮限界へ到達した場合は、領域を一時的にわずかに拡大し、円盤へ小さな乱数変位を与えてから再圧縮するescapeを試した。escape後に元の候補より小さい L まで圧縮できた場合だけ採用する。これは、多スタート探索で得た有望な局所解を捨てず、そこからさらに別の局所解へ移る試みである。

初期摂動と同様に乱数を利用するため、隙間の位置を理解して動かす方法ではない。しかし、実験ではescapeによって改善する例を確認できた。乱数に頼るという全体方針を、初期探索だけでなく最終候補の再探索にも一貫して利用した。

1.5 方針を支える圧縮処理

多数試行を成立させるため、各試行自体も高速化した。サンプルの固定刻みの最急降下法をFIRE（Fast Inertial Relaxation Engine）法へ置き換え、粗い圧縮、境界探索、最終精密化でパラメータを切り替えた。通常設定で失敗した場合だけ保守的な設定で再試行するハイブリッド方式とした。

円盤の接触判定には動的なセルリストを使い、力計算などをOpenMPで並列化した。圧縮が失敗した後は成功側と失敗側の間を二分探索し、試行時間の実測値から残り時間内に完了できる圧縮幅を選んだ。最後に、常に検証済み状態を保存して失敗時に戻すことで、多数の試行を自動で安全に進められるようにした。これらは独自戦略そのものではなく、多スタート探索へ計算時間を回すための基盤として位置付けた。

1.6 実験結果と限界

初期摂動、初期充填率、時間配分、escapeの強さなどを一度に一項目ずつ変更して比較した。小さな改善しか期待できない調整より、平均 L を 10^{-2} 以上改善できる可能性がある項目を優先した。最終版では初期充填率を0.75、初期摂動を円盤直径の0.10倍とした。

提出版と同じソース、ジョブスクリプト、新しく生成した8問の入力を使った本番形式のテストでは、修正版採点プログラムを最適化オプションなしでコンパイルして検証し、8問すべてが合格した。このテストでの L の合計は425.2681、平均は53.1585、全体の処理時間は594.017秒であった。

多数試行は実際に良い局所解を発見できた一方、最終候補にならなかった試行へ使った計算資源は直接は結果に残らない。また、乱数によるescapeは隙間の位置を理解して動かすものではないため、局所構造を利用した方法より非効率である可能性がある。

今後は、局所密度や接触構造を調べ、高密度領域と低密度領域の間で円盤を意図的に再配置するなど、隙間を埋めやすい方向へ誘導するescapeが考えられる。今回の方法は、局所解の存在と余った並列計算資源に着目した実用的な戦略であり、その有効性と限界の両方を実験から学ぶことができた。

2. アンケート

(a) コンテストの感想

これまで解いたことのない、一般的な競技プログラミングではあまり見かけない種類の問題について考えることが新鮮で、面白かった。決まったアルゴリズムを正確に実装するだけでなく、問題の解法そのものを考え、さらに他チームがどのような方針を採るかを推測して、どこで差を付ければ勝てるのかを考える過程を楽しめた。

ただ圧縮するだけでは、隙間が残っているのに動かなくなる局所解へ入ることに気付いた。私たちは、悪い局所解そのものを巧妙に避ける代わりに、初期条件を何度も変えて試行し、良い局所解へ到達したものを採用する「数撃ちゃ当たる戦法」（初期摂動を用いた確率的多スタート探索）を考えた。効率面には改善の余地があるものの、発想を逆転させたこの方法で実際に良い結果が得られたことが面白かった。

チームメイトの結果やDiscord上の雑談も、自分たちの実装を評価する参考になった。勝負が小数点第一位や第二位で決まるのではないかという意見から、平均 L は53台で競われると予想した。一方で、私たちの一部の結果は概算した理論値を下回っており、局所解の選択が成績へ大きく影響するという考えを強めた。

参加を通じて並列プログラミングへの理解を深められた。OpenMPで一つの処理を細かく並列化する場合と、MPIで独立した仕事をプロセスに分けて候補を通信する場合を実際に比較し、仕事の性質に応じて並列化方法を選ぶ重要性を体験として学べた。

チーム内で方針と進捗を共有できたことも、モチベーションの維持につながった。

(b) AIの活用

YES

チームでは、OpenAI Codex（GPT-5系）、ChatGPT、Claudeを使用した。どのようなアプローチが考えられるかの相談、プログラムの分析と改善案の検討、C++の実装とデバッグ、実験計画の整理、実験結果の検証・比較・考察、レポート原稿の整理に利用した。

AIは大量のコードやログを整理し、複数の改善案を具体的な実験へ落とし込むうえで役立った。一方で、AIはときどき誤った理解や提案をするため、提案を採用するか、どの方針で戦うかという最終判断は人間が行った。AIの回答をそのまま採用するのではなく、実験結果と大会規則を確認しながら利用した。

(c) 富岳の使い勝手について

初めて使う仕組みが多く、初日から2日目の午前までは操作に慣れるために時間を要した。特にSSH鍵の設定には苦労したが、リモート計算機へ安全に接続する仕組みを学ぶよい機会になった。

一方で、サポート資料が豊富であり、資料を読めば多くの問題を解決できた。ジョブスケジューラや計算資源の割り当て方から、多くの利用者が巨大な計算資源を公平かつ効率よく共有するために、よく考えられた設計になっていることが分かり、感心した。

実行までに必要な手順が多い点は、最初は少し面倒に感じたが、慣れてからは大きな問題ではなかった。改善してほしい点として、Open OnDemandのHistory画面では、ページを再読み込みしないとジョブの状態が更新されないことが挙げられる。自動的に状態が更新されれば、より使いやすくなると思った。